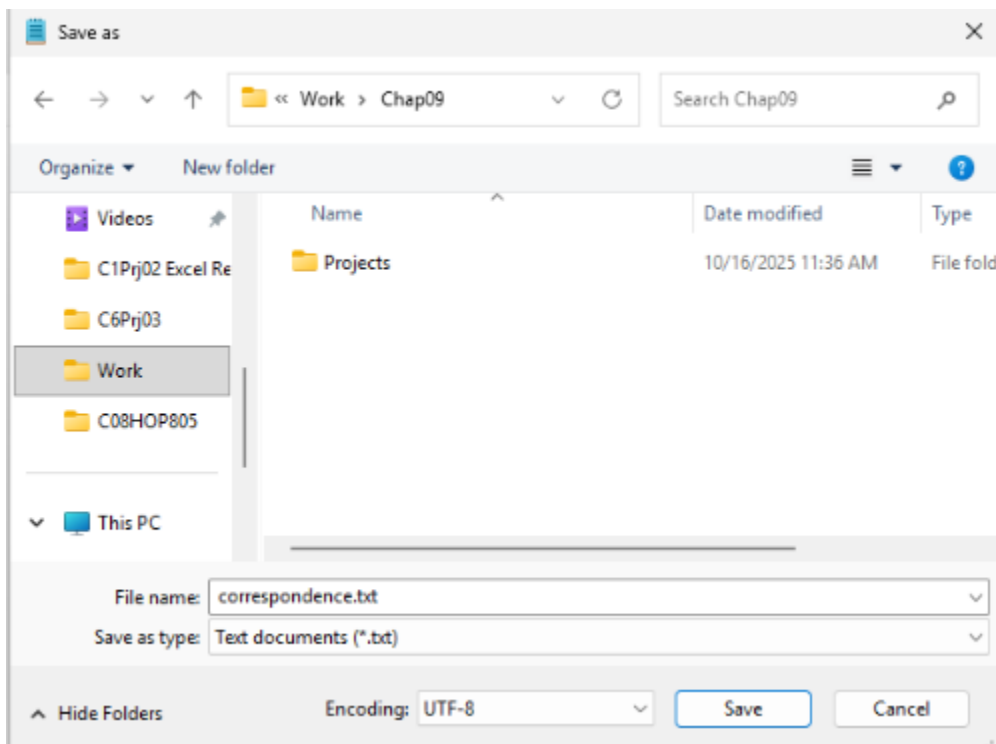
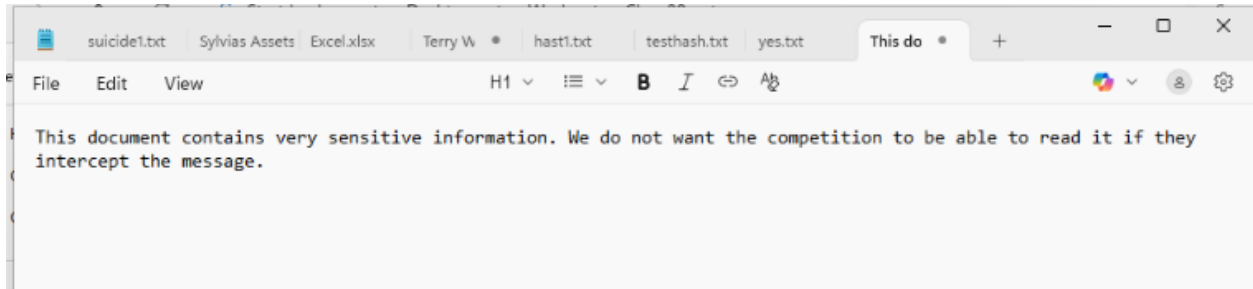
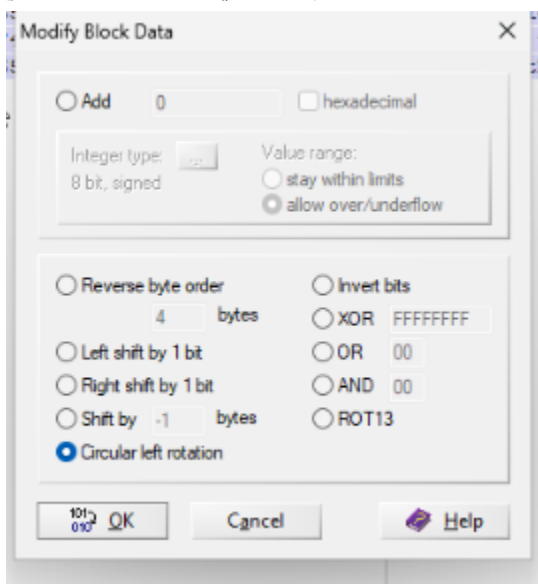
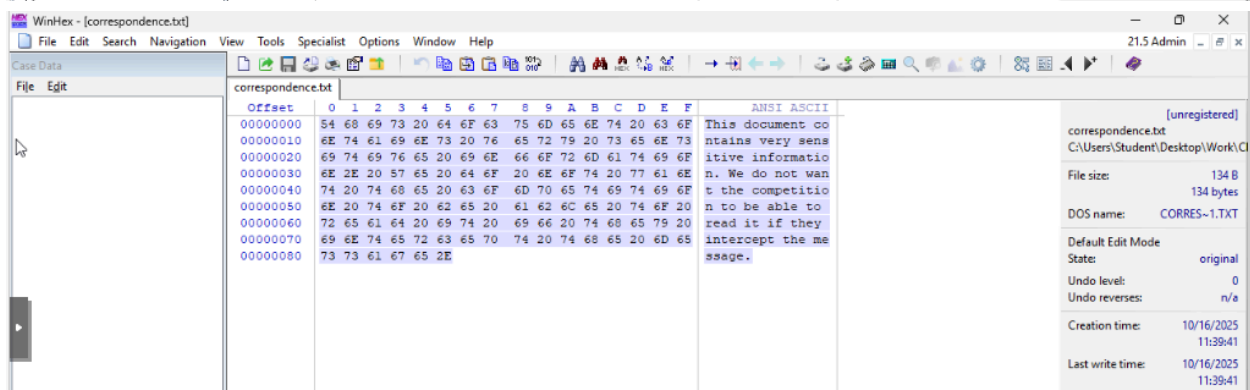
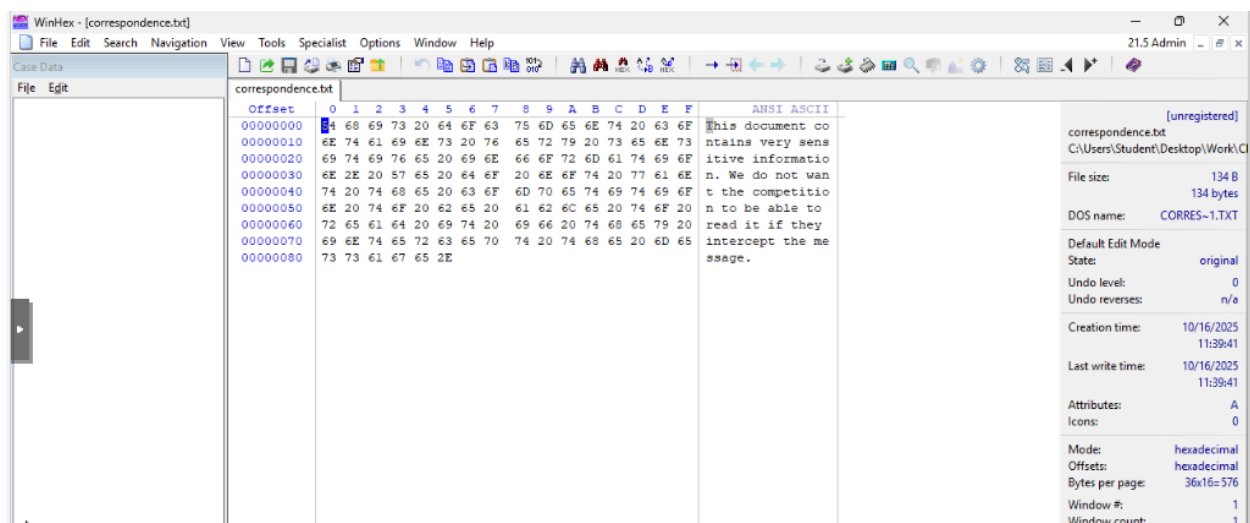


9-1 Hands on Project

Jack Irvin

10/16/2025





File name: Save

Save as type: Cancel

WinHex - [correspondence2.txt] 21.5 Admin

File Edit Search Navigation View Tools Specialist Options Window Help

Case Data

File Edit

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	ANSI ASCII
00000000	54	68	69	73	20	64	6F	63	75	6D	65	6E	74	20	63	6F	This document co
00000010	6E	74	61	69	6E	73	20	76	65	72	79	20	73	65	6E	73	ntains very sens
00000020	69	74	69	76	65	20	69	6E	66	6F	72	6D	61	74	69	6F	itive informatio
00000030	6E	2E	20	57	65	20	64	6F	20	6E	6F	74	20	77	61	6E	n. We do not wan
00000040	74	20	74	68	65	20	63	6F	6D	70	65	74	69	74	69	6F	t the competitio
00000050	6E	20	74	6F	20	62	65	20	61	62	6C	65	20	74	6F	20	n to be able to
00000060	72	65	61	64	20	69	74	20	69	66	20	74	68	65	79	20	read it if they
00000070	69	6E	74	65	72	63	65	70	74	20	74	68	65	20	6D	65	intercept the me
00000080	73	73	61	67	65	2E											ssage.

correspondence1.txt | correspondence1.txt | correspondence1.txt | correspondence1.txt | correspondence2.txt

File size: 134 B
134 bytes
DOS name: CORRES~3.TXT
Default Edit Mode: original
State: original
Undo level: 0
Undo reverses: n/a
Creation time: 10/16/2025 12:08:42
Last write time: 10/16/2025 12:08:28
Attributes: A
Icons: 0
Mode: Text

In this lab I worked with a text file called correspondence.txt to test how bit-shifting can hide data. The file had a short sensitive message. Using WinHex I applied the circular left rotation option to shift the bits in each byte, saved the scrambled file as correspondence1.txt, and confirmed the text was unreadable. Reapplying the same rotation eight times restored the ASCII text exactly, and I saved the recovered file as correspondence2.txt. That shows the method is reversible and works predictably for this type of data.

Even though bit-rotation hides the content, it's not a reliable encryption method. The main problem is that it's uniform: every byte is shifted the same way, so a clear pattern is introduced across the whole file. If an attacker suspects bit manipulation, they can look for that pattern and reverse it. In practice a knowledgeable person or analyst can identify the telltale signs of rotation by comparing known file structures or using simple statistical checks. For example, ASCII text has recognizable byte distributions; when those distributions are rotated, the change is detectable. Once identified, the rotation can be undone easily, either with tools designed for binary editing or with WinHex itself by applying the opposite rotation the appropriate number of times.

Because the method is so simple, any moderately skilled person can crack it. Automated tools, scripts, or even trial-and-error in a hex editor will reveal the rotation amount. There's no key, no randomness, and no cryptographic strength, just a fixed transformation that can be analyzed and inverted.

Overall, bit-rotation should not be used to protect real sensitive data. It's a great way to manipulate data easily and for fun, but not a security solution: if someone can identify the manipulation, they can almost certainly recover the original information.

